

Take-Home Assignment: Production Interview Agent with Feedback Loop

Goal

Build an end-to-end, deployable conversational interview agent and a production feedback loop that lets human evaluators improve the agent's prompts over time.

Time

You have **7 days** to complete this assignment. Prioritize a working end-to-end flow over extra features.

What You Will Build

1. Conversational Interview Agent

A conversational agent that conducts a short technical interview. The agent selects one skill from a fixed list, asks 3 questions for that skill, and for each question evaluates the candidate's response, clarifying or probing when it judges the answer to be incomplete or ambiguous.

The agent should be **stateful and multi-step** (e.g., ask → evaluate → clarify → next question) with clear state management and transitions, not a single linear LLM call. Whether you implement this as a single agent with a state machine, multiple cooperating agents, or another architecture is your design decision, justify your choice in the README.

The agent must be deployable and callable by a client (e.g., REST API, WebSocket, or another protocol). The interview flow (welcome, questions, wrap-up) and client interaction model are up to you.

2. Production Feedback Loop for Human Evaluation

A system that supports human evaluators reviewing completed interview conversations. Evaluators should be able to:

- **Discover** conversations (e.g., by ID, date, skill, or status).
- **Rate** conversations (e.g., overall quality, fairness, relevance).
- **Flag issues** (e.g., inappropriate question, weak evaluation, bias).
- Optionally, **compare** two conversations or agent versions side by side (A vs B).

How evaluators interact with the system (CLI, API, simple web UI, scripts) and how feedback is stored is your design decision. The key requirement is that the feedback is persisted and accessible, it should be possible to retrieve past ratings and flags for a given conversation or across conversations.

3. Closing the Loop: Prompt Update Path

A mechanism to update the prompt instructions used by the interview agent (e.g., system prompts, evaluation criteria, clarification rules).

How prompts are stored, versioned, and loaded is your design decision (e.g., config file, database, parameter store, API). At minimum, it should be possible to tell which prompt version was used for a given conversation.

Skills

Use the following skill list for the interview:

- Problem Solving
- Communication
- Collaboration & Teamwork

What We're Looking For

Required (Must Have)

These are the minimum requirements for a complete submission.

1. **Interview agent with stateful flow:** ask → evaluate → clarify → next question, for 3 questions in one skill. The agent must handle at least one full interview end to end.
2. **Deployable interface:** the agent is callable by a client (API, WebSocket, or equivalent). It runs and can be interacted with.
3. **Feedback submission:** evaluators can submit ratings and/or flags for a completed conversation, and that feedback is persisted.
4. **Conversation retrieval:** evaluators can look up and view a completed conversation (e.g., by ID).
5. **Prompt loading from configuration:** the agent reads its prompt instructions from a configurable source (file, DB, env var, etc.), and changing that source changes the behavior of new conversations.
6. **Tests:** at a minimum, include tests for core logic, agent state transitions, feedback submission, and prompt loading. The type and depth of testing (unit, integration, end-to-end) is up to you, but the submission should demonstrate that the system works as intended and that key paths are verified.

7. **README:** how to set up and run the agent, how to submit feedback, how to update prompts and verify they're in use. Document any assumptions you made.
8. **Demo walkthrough:** a script, Makefile target, or set of commands that walks a reviewer through the full flow: run an interview → view the conversation → submit feedback → update a prompt → run a new interview with the updated prompt. This should be simple to execute without deep knowledge of your codebase.

Expected

These demonstrate production thinking and are expected from a senior candidate.

1. **Containerized deployment** (e.g., Docker / Docker Compose) so the reviewer can run the full system with a single command.
2. **Conversation discovery and filtering:** evaluators can search or filter conversations (e.g., by date, skill, status, or whether they've been reviewed).
3. **Prompt versioning:** prompts are versioned, and each conversation records which prompt version was used.
4. **Error handling and resilience:** the system handles LLM failures, malformed input, and unexpected states gracefully (e.g., retries, timeouts, meaningful error messages).
5. **Basic observability:** health check endpoint, structured logging, or one or two key metrics (e.g., interview completion rate, average response latency).

Bonus (Nice to Have)

These are not expected but will strengthen a submission.

1. **A/B comparison:** evaluators can compare two conversations (or two prompt versions) side by side and submit a preference.
2. **Feedback aggregation:** a view or script that summarizes feedback across conversations (e.g., average ratings per skill, most common flags, trends across prompt versions).
3. **CI/CD pipeline:** automated build, test, and deploy (e.g., GitHub Actions, similar).
4. **Prompt update API:** an API endpoint (rather than manual config editing) to update and activate new prompt versions.
5. **Automated prompt improvement suggestions:** using aggregated feedback to suggest or draft prompt changes.

Reviewer Experience

Please optimize your submission so a reviewer can clone the repo and run the full system with minimal effort.

Assumptions

- **LLM provider:** use any LLM provider you prefer (OpenAI, Anthropic, open-source, etc.). Include clear instructions for configuring the API key. If you'd like to support a mock/stub mode for reviewers who don't have a key, that's a nice touch but not required.
- **Stack:** no constraints on language, framework, or infrastructure. Use what you're most productive with.
- **Scope:** you may make and state reasonable assumptions (e.g., number of evaluators, how often to clarify, feedback schema, how skills map to questions). Document them briefly in the README.
- **UI:** a frontend is not required. APIs, CLI tools, or scripts are perfectly acceptable for all interactions. If you do build a UI, it will not be evaluated on visual polish, only on functionality and clarity.